

CS3160 Lab 5

The Single-Cycle Processor

What you will do today

You now have a memory that holds a program, and a decoder that can say what each word in the memory means. By end of today's session, your simulator will execute real programs and report how many instructions it took, and what it did to memory.

This is the first week your simulator is a processor, and future labs will improve the performance of this simulated processor. The single cycle processor model gives the semantics of a program execution, and therefore, will serve as the base model, all future enhancements is checked against.

1 The datapath

The processor is simulated using seven functions and a loop. An overview of the tasks to be done is given below:

Stage	Function	Yours?
fetch	<code>stage_fetch()</code>	given
decode	<code>decode()</code>	week 4
operands	<code>stage_operands()</code>	given
execute	<code>stage_execute()</code>	yes
next pc	<code>next_pc_of()</code>	yes
memory	<code>stage_memory()</code>	yes
writeback	<code>stage_writeback()</code>	yes
the ALU —	<code>alu_apply()</code> and <code>branch_taken()</code>	yes
the run loop —	<code>run_single()</code>	yes

Your work will be confined to three files: `src/alu.cpp`, `src/processor.cpp` and `src/processor_single.cpp`. The place where you are required to edit is marked `TODO(week 5)`. The associated comment tells exactly what is to be done and which helpers exist for it. **Read those comments** because this handout does not repeat them.

2 Part 1: the ALU

`src/alu.cpp` has two halves: `alu_inputs()` and `alu_apply()`.

`alu_inputs()` is the operand multiplexers: given a decoded instruction, it decides which two values reach the ALU. For an `add` instruction, the two two registers are output. Similarly, for an `addi` instruction, a register and the immediate; for an `lui` instruction, zero and the immediate;

for `auipc` and `jal` instructions, the PC and the immediate are output. This function is already written for you. Read it before you write anything else because understanding this function will help you implement rest of the processor. This function helps in implementing `alu_apply()` because an address calculation, an `addi` and a jump target, etc become a simple “add these two numbers”.

`alu_apply()` is the ALU itself — one switch-case over `Op`, two 32-bit patterns in, one out. Write `OP_ADD` and `OP_SUB`, run `make test`, and work down the list of failures: each unit test is named for the operation it checks.

CAUTION: The four cases C++ gets wrong

Most operations are the C++ operator with the same name. Four are not, because RISC-V defines a result where C++ says *undefined behavior*, and on those four your compiler is not a guide:

Case	RISC-V says	In C++
<code>div</code> , <code>divu</code> by zero	all ones (-1)	undefined
<code>rem</code> , <code>remu</code> by zero	the dividend	undefined
$-2^{31} \div -1$	-2^{31}	undefined (overflow)
-2^{31} <code>rem</code> -1	0	undefined

Each needs an explicit test before you divide. There is no trap and no exception: RISC-V has no divide-by-zero fault at all, so a program that divides by zero gets -1 and keeps running. The unit tests check all four.

► *Write these as tests on the bit patterns and cast to `i32` only for the arithmetic itself. `as_signed()` in `common.h` is how you say you meant the signed reading of a pattern. Everything in the register file is a `u32`; nothing carries a sign around in its type, because the hardware does not either.*

For the shifts, `shift_amount()` is sitting just above the `TODO`: only the low five bits of the amount count, since a 32-bit register cannot move more than 31 places, and the specification says the upper bits are ignored.

`branch_taken()` is the other half: given the two registers a branch compares, is it taken? There are six operations to handle, and ensure to handle the signed comparisons — `blt`/`bge`, the unsigned comparisons — `bltu`/`bgeu` appropriately. NOTE that `0xFFFFFFFF` is below zero under one reading (signed case) and the largest value there is under the other (unsigned case).

3 Part 2: the four stages

These are in `src/processor.cpp`, in the order the loop calls them.

3.1 `stage_execute()`

Put the ALU to work: `alu_inputs()` to pick the operands, `alu_apply()` to combine them, `branch_taken()` for the second field. Only branch instructions have an opinion about `taken`.

3.2 next_pc_of()

There are three places from where the next instruction can come from (in the order given):

1. A **jump** goes wherever the ALU computed. This is the payoff for `alu_inputs()` treating `jal` as “PC plus immediate” and `jalr` as “register plus immediate”: the target is already sitting in `e.result`.
2. A **taken branch** goes to its own `pc` plus its immediate — not to `e.result`, which for a branch is meaningless.
3. **Everything else** goes to `pc + 4`.

► *The immediates were sign-extended for you back in week 4, so a backward branch is `d.pc + d.imm` with no special case: unsigned arithmetic wraps, and adding `0xFFFFFFFF8` is subtracting 8.*

3.3 stage_memory()

Only loads and stores touch memory; for everything else this stage does nothing and says so. The address is already computed in the ALU (its result). So only three things are left to compute:

- **The width.** `mem_size()` gives 1, 2 or 4 bytes, and `cpu.dmem->read()` and `->write()` take it.
- **The sign.** A load of less than a word arrives in the low bits. `lb` and `lh` fill the rest with the sign; `lbu` and `lhu` fill it with zeroes. `mem_signed()` says which, `sign_extend()` does it. The week-3’s `ram_read()` zero-extends everything, which was right then: whether the sign should be filled in, depends on the *instruction*, and has to be handled here.
- **How the program stops.** `crt.S` ends by storing to the address in `cpu.cfg->tohost`. When that store happens, set `cpu.halted` and record the exit status.

► *If you miss handling the `tohost` store, your simulator will never terminate. A safety mechanism that is implemented is to run terminate the simulation once a preset instruction budget hits. The harness reports such a program as failing to finish. If `make test` tells you a program did not stop, look for this first.*

3.4 stage_writeback()

There are two tasks to be done here. Write the result to `rd` if this instruction writes one: `writes_rd()` says whether it does, `writeback_value()` picks between the ALU result and what a load brought back, and `reg_write()` is the one that knows `x0` stays zero whatever you hand it.

The second task is to count. Every instruction that reaches here has retired, and loads, stores, jumps and branches each have their own counter. `taken_branches` is the one to think about (see §5 for more details).

Finish with `trace_retire()`, which writes the [OUT] line the tests compare against. It is written for you; call it with the `next_pc` you were handed rather than with `d.pc + 4`, or every branch in the trace will look like it fell through.

4 Part 3: the run loop

`src/processor_single.cpp`. One instruction per iteration, seven stages in order, then move the PC to where `next_pc_of()` said and stop when `cpu.halted` goes true. Bound the whole thing by `cfg.num_insts` so a program that never terminates stops being a problem after a million instructions.

The cycle count is one line. This machine takes **one cycle per instruction**, plus whatever the memory made it wait for — `cycle_cost_serial(fetch latency, data latency)`. With the default ideal memory both latencies are zero, every instruction costs one cycle, and `cpi` comes out exactly 1.0000.

5 Two counts of the same thing

Your simulator reports the number of times control went somewhere else (not straight-line code) *twice* (by two different code-paths), and on one of the five test programs, these two counts differ. This is deliberate, and you should not fix it.

pc_redirects — measured.

Counted in your run loop, using `redirects_pc()`, which reads the branch’s own verdict out of the `Executed` struct. It is not a report of the decision; it *is* the decision, written down as it is made.

taken_branches — reconstructed.

Counted in `stage_writeback()`, which has no `Executed` to consult. It is handed `next_pc` and works backwards: a branch was taken if `next_pc != pc + 4`.

For every branch a compiler will ever emit, those agree. The reconstruction has exactly one blind spot, and `10-branch-offset4.s` is written to find it:

```
beq    zero, zero, .+4          # always taken -- to the next instruction
```

`beq zero, zero` is always true, and an offset of `+4` makes the target the instruction it would have fallen through to anyway. The branch *is* taken. The reconstruction looks at `next_pc`, finds `pc + 4`, and concludes it was not. Your `stats.json` for that program should read:

<code>branches</code>	8	six in the program, two in <code>crt.S</code>
<code>taken_branches</code>	1	the four self-targeting ones are invisible
<code>jumps</code>	4	
<code>pc_redirects</code>	9	4 branches + 4 jumps + 1: every real one

Five taken branches happened; the reconstruction found one. **The tests expect both of these numbers**, so a `taken_branches` that has been “corrected” to 5 fails. The point is not the encoding, which no compiler emits. The point is that a number worked out afterwards from something else is an argument, and arguments have cases they do not cover — and you cannot tell which numbers those are by looking at `stats.json`, only by knowing how each one was obtained.

6 Checking your work

```
make          # build build/sim
make test     # build, then unit tests, then programs
make unit     # just the unit tests, quieter while you work on one
               stage
```

`make test` is the same checker used to grade your work. There are three tiers: it compiles, then 32 unit tests that call one stage at a time, then the five programs, where your simulator's trace and statistics are compared against ours instruction by instruction.

`docs/running-your-simulator.md` in your lab folder covers running the simulator by hand, the options, and — worth reading before you need it — how to read a trace failure and tell which stage caused it. `docs/RISCV_CARD.pdf` is the instruction reference.

An order that works

The unit tests are the map. A do-nothing starter passes 4 of the 32 (they are tests that happen to be true of a stub), so anything above that is progress.

1. `alu_apply()`: add and sub, then the logical operations, then the shifts, the comparisons, and the M extension last.
2. `branch_taken()`, and then `stage_execute()`, which is three calls once the ALU works.
3. `next_pc_of()` and `stage_memory()`.
4. `stage_writeback()`, then `run_single()`.

No program can run until the loop exists, so the program tier stays at zero until the very end and then moves all at once. Do not read that as being stuck — read the unit count.

► *The starter compiles without a single warning, and `-Wall -Wextra` is on. So every warning you see is one you introduced, and in C++ a warning is very often the bug you were about to spend an hour on. Fix them as they appear.*

7 What to hand in

Your folder is collected at the end of the session and **only what is in it at that moment is graded**:

1. `details.txt` — name, roll number, machine number. If this is blank, your submission cannot be matched to you.
2. `src/alu.cpp`, `src/processor.cpp` and `src/processor_single.cpp`.